دليل مكتبة fast-check (v4.10.2) الميداني

مرجع اختبار الخصائص البرمجية (Property-Based Testing) — نظام البنشر Pro

`v4.10.2`
مثبتة بالخادم

النمط الهندسي	تغطية السيناريوهات	الميزة الحصرية	الأمان المحاسبي
Property Testing	**100+ حالة بالثانية**	**التقليص الذكي (Shrink)**	**صفر أخطاء خفية**

1 ما هي fast-check؟ ولماذا نحتاجها؟ (WHAT & WHY)

المفهوم باختصار: بدلاً من تجربة الفاتورة برقم أو رقمين تضعهما بيدك (الاختبار اليدوي)، تقوم `fast-check` بقصف الكود بآلاف الأرقام والمدخلات العشوائية والمتطرفة (أصفار، سالبة، كسور لا نهائية، مبالغ هائلة بالريال) لتتأكد أن معادلاتك المحاسبية لا تنكسر تحت أي ظرف.

الاختبار التقليدي (Example-Based)

- تفحص: `فاتورة(10,000) - خصم(1,000) = 9,000` ✅
- **المشكلة:** إذا أدخل الكاشير خصماً `10,000.5` أو قيمة سالبة بالخطأ، قد ينتج خلل مالي لا تكتشفه إلا في الميدان.

اختبار الخصائص (fast-check)

- تعطي المكتبة قانوناً عاماً: *"مهما كانت قيمة الفاتورة والخصم، الصافي لا ينزل تحت الصفر أبداً"*.
- **القوة:** تفحص 100 إلى 1,000 سيناريو عشوائي في ثانية واحدة وتكشف الثغرة فوراً إن وُجدت.

2 طرق التثبيت والاستدعاء (Installation & Import)

التثبيت عبر سطر الأوامر (Terminal)

```
# تثبيت كحزمة تطوير في خادم البنشر
npm install --save-dev fast-check@4.10.2

# أو باستخدام pnpm / yarn
pnpm add -D fast-check@4.10.2
yarn add -D fast-check@4.10.2
```

(ملاحظة: نضعها في devDependencies حتى لا تزيد حجم سيرفر العميل النهائي).

طريقة الاستدعاء البرمجي في الكود

```
// 1. في خادم Node.js / CommonJS (المعتمد في السيرفر):
const fc = require('fast-check');

// 2. في واجهة React / Vite (ES Modules):
import fc from 'fast-check';
```

✅ مثبتة بالفعل في مجلد `server` وجاهزة للاستخدام الفوري.

3 الأركان الأربعة الأساسية لعمل fast-check

1 المولدات (Arbitraries - `fc.*`)

توليد قيم عشوائية حسب ضوابط معينة (أرقام، نصوص، مصفوفات، كائنات).

`fc.integer({ min: 1, max: 500 })`

2 الخاصية أو القانون (Property - `fc.property`)

تستقبل البيانات المولدة وتمررها للدالة للتأكد من انطباق القاعدة المنطقية.

`fc.property(fc.float(), (val) => ...)`

3 محرك الفحص والتشغيل (Runner - `fc.assert`)

يشغل الدورة مئات المرات متتالياً ويوقف الاختبار ويعطي تقريراً فورياً عند الخلل.

`fc.assert(fc.property(...))`

4 التقليص الذكي للخطأ (Shrinking Engine)

إذا ظهر خطأ عند رقم ضخم مثل `892,410.5-`، تصغره تلقائياً إلى `1-` لتعطيك السبب المباشر.

أقوى ميزة لاكتشاف السبب الجذري

تابع الصفحة 2: أمثلة برمجية حية من نظام البنشر + جدول المولدات وإعدادات التشغيل المتقدمة ←

4 3 نماذج تطبيقية حية من صلب نظام "البنشر Pro"

1. فحص دقة تقريب المبالغ بالريال اليمني (Currency Rounding Guard)

التأكد أن دالة `roundMoney` لا ترجع أبداً `NaN` أو رقماً لا نهائياً تحت أي قيمة عشرية:

```
const fc = require('fast-check');
const { roundMoney } = require('../utils/mathPrecision');

fc.assert(
  fc.property(fc.double({ noNaN: true }), (amount) => {
    const result = roundMoney(amount);
    return typeof result === 'number' && isFinite(result);
  }),
  { numRuns: 300 } // تشغيل 300 سيناريو عشوائي
); // النتيجة: NaN ✅ تم اجتياز كافة الحالات بدون أي خطأ
```

2. صمام أمان الخصومات في الفواتير (Discount Overdraft Guard)

إثبات قانون: صافي الفاتورة لا يمكن أن ينزل تحت الصفر مهما كان الخصم عشوائياً أو مبالغاً فيه:

```
function calcNetTotal(subtotal, discount) {
  return Math.max(0, subtotal - Math.max(0, discount));
}

fc.assert(
  fc.property(
    fc.float({ min: 0, max: 5000000 }),     // إجمالي الفاتورة حتى 5 مليون ﷼
    fc.float({ min: -1000, max: 10000000 }), // خصم عشوائي (بما فيها خصم أكبر من الإجمالي)
    (subtotal, discount) =>  {
      const net = calcNetTotal(subtotal, discount);
      return net >= 0 && net <= subtotal; // الصافي محصور دائماً بين الصفر والإجمالي
    }
  )
);
```

3. حساب متوسط التكلفة المرجح للمخزون (WAC Inventory Verification)

التأكد أن تكلفة لتر الزيت الجديد تقع رياضياً بين التكلفة القديمة وسعر الشراء الجديد دائماً:

```
const { calcWAC } = require('../utils/mathPrecision');

fc.assert(
  fc.property(
    fc.integer({ min: 1, max: 200 }), fc.float({ min: 1000, max: 15000 }), // مخزون وتكلفة حالية
    fc.integer({ min: 1, max: 200 }), fc.float({ min: 1000, max: 15000 }), // كمية وتكلفة وارد جديد
    (currQ, currC, inQ, inC) =>  {
      const wac = calcWAC(currQ, currC, inQ, inC);
      return wac >= (Math.min(currC, inC) - 0.01) && wac <= (Math.max(currC, inC) + 0.01);
    }
  )
);
```

5 البطاقة المرجعية السريعة (Cheatsheet & Runner Options)

المولد (Arbitrary)	استخدامه في نظام البنشر
`fc.integer({ min, max })`	كميات الفلاتر والإطارات وعداد الكيلومترات.
`fc.float({ min, max })`	لترات الزيت المسكوبة ومبالغ الخصومات.
`fc.string({ minLength })`	أرقام لوحات السيارات وأسماء العملاء.
`fc.record({ ... })`	بناء فاتورة أو حركة مخزنية كاملة ببيانات عشوائية.

خيار التشغيل (Option)	الفائدة والأهمية
`{ numRuns: 1000 }`	زيادة عدد التجارب العشوائية للتدقيق الفائق قبل الإطلاق.
`{ seed: 987654 }`	تثبيت العشوائية لتكرار نفس السيناريو الخاطئ وفحصه.
`{ endOnFailure: true }`	إيقاف الفحص فور العثور على أول خطأ لتسريع التصحيح.
`{ timeout: 2000 }`	منع تعليق النظام في الدوال التي قد تحتوي حلقات لا نهائية.

🎯 **الخلاصة الهندسية:** مكتبة `fast-check` هي صمام الأمان الذي يحمي نظام "البنشر Pro" من الانهيارات المفاجئة والمشكلات الحسابية المعقدة في الميدان. تمنحك الثقة المطلقة في أن الكود لا "يبدو أنه يعمل" فقط، بل هو سليم ومحصن رياضياً بنسبة 100%.